Computer Science & IT



Compiler Design

Syntax Analysis (Parser)

Lecture No. 06



By- Vishal Sir

Recap of Previous Lecture



Topic FOLLOW set computation

Slide

Topics to be Covered



- Topic: Homework Questions of FIRST & FOLLOW
- Topic: Top-down Parser
- Topic: LL(1) Parsing Table construction
- Topic: LL(1) Parsing Algorithm



Topic : First & Follow

H.W

	First	Follow
S→aBDh	{a}	{$}
B→cC	{c}	first(Dh) = {g, f, h}
C→bc / ε	{b, ε}	follow(C) = follow(B) = {g, f, h}
D→EF	first(EF) = {g, f, ε}	follow(D) = first(h) = {h}
E→g/ ε	{g, ε}	follow(E) = {f, h}
F→f/ ε	{f, ε}	{h}

(first(F) − {ε}) ∪ follow(D)

Slide



Topic : First & Follow

	First	Follow
S → AaAb / BbBa	first(AaAb) ∪ first(BbBa)	{$}
	{c, a} ∪ {d, b} = {a, b, c, d}	
A → c / ε	{c, ε}	
B → d / ε	{d, ε}	

S → AaAb → first(aAb) = {a}

S → AaAb → first(b) = {b}

= {a, b}

follow(B) = { S → BbBa, S → BbBa }

= {b, a} = {a, b}

Slide



Topic : First & Follow

$L \to SL'$

(First(L') − {ϵ}) ∪ follow(L) ⇒ {)}

Follow

First

Follow		First	Follow
{$, ;,)}	S → (L)/a	{(, a}	{$, ;,), same as Follow(L)} = {$, ;,)}
{)}	L → SL'	First(SL') = {(, a}	{)}
{)}	L' → ; SL' / ε	{;, ϵ}	{)}

L' → ; S L'

(First(L') − {ϵ}) ∪ follow(L')

Slide



Topic : First & Follow

$\{d,g,h,\epsilon\} \cup \{h,b\} \cup \{g,a\}$ First

$S \rightarrow ACB / CbB / Ba$ $= \{d,g,h,b,a,\epsilon\}$

$A \rightarrow da / BC$ $= \{d,g,h,\epsilon\}$

$B \rightarrow g / \varepsilon$ $= \{g,\epsilon\}$

$C \rightarrow h / \varepsilon$ $= \{h,\epsilon\}$

Follow

$\{\$\}$

$\{h,g,\$\}$

$\{\$,a,h,g\}$

$\{g,\$,b,h\}$

Slide



Topic : Top-down Parser

- If any parser uses top-down approach then it will start the construction parse tree from start symbol, and it uses Leftmost derivation in the Construction of Parse tree.
- Top-down parser can see only one symbol at a time of input string from left to right.

Slide



Topic : Type of Top-down Parsers

If we have made any incorrect move, then we can go back and explore other alternatives

Type of Top-down Parser

Top-down Parser (with backtracking)

We can { Try all Possible options } (i.e. Brute force Method)

eg: Recursive descent parser with backtracking

(Not table driven Parser)

Top-down Parser (Without back-tracking)

backtracking is not allowed

Predictive Parser

Recursive -descent Parser with no back-tracking

Non-Recursive Predictive Parser { Also known as LL(1) parser }

{ Table driven Parsers }

w.r.t. GATE

Slide



Topic : LL(1) Parser

①

it is a Top-down parser (without backtracking)

a.k.a, Non-Recursive Predictive Parser

because it is table driven

L L (1) Parser

- 1 look-ahead symbol.
- Leftmost derivation
- Left to Right Scanning

If there are two or more productions corresponding to any non-terminal of the grammar then we need to predict the best production based on lookahead symbol of the input string. ∴ it is called Predictive Parser.

Slide



Topic : L L (1) Parser

A grammar Can be parsed using LL(1) parser only if

(i) Grammar is unambiguous grammar

(ii) Grammar does not Contain any left recursive Production

(iii) Gramma does not contain Common-Prefix Problem.

Slide



2 mins Summary

Topic First & Follow Concept

Topic LL(1) Parser table construction

Topic LL(1) Parsing Algorithm


THANK - YOU
Slide